

RBE 4701 - Project 1 Report

Team 05 - Brendan Bacon, Rowan Michaud, & Jake Claybrook

For the first project of RBE-4701: Artificial Intelligence in Robotics, we were tasked with coding an AI to play the game bomberman. Our character was limited to two actions, move and place bombs. The rest of the methods we were provided helped us with analyzing the map, enemies, and most importantly the exit. The goal was for our AI to reach the exit on 50% or more of all survival variants we were provided.

There are 4 main states that we coded our bomberman character to be in: HIDE, BOMB, EVADE, and SEEK_EXIT. The AI checks for states in that exact order, with it first entering the HIDE state if a bomb or explosion exists in the world. If not, it then checks if there is no path to the exit or if it has not made progress for some time while not currently being under threat by any monsters, and if so it enters the BOMB state. Next it checks if it is currently being threatened by any monsters, determined by comparing its current threat level to a threshold set, and if it is, the AI enters the EVADE state. If none of the conditions for these three states are met, the AI simply enters the SEEK_EXIT state. If the AI is in the SEEK_EXIT state, it simply follows the optimal path calculated with the current world state to reach the exit. If it is in the BOMB state, it drops a bomb. For both the EVADE and HIDE states, the AI runs a search algorithm to find the best move given the current world state.

Our AI uses A* to calculate the optimal path from its current location to the exit when seeking the exit, with the only edge case being if A* does not return a path longer than just the starting square, in which case it falls back to the search algorithm. The search algorithm used is a combination of a hard safety filter and expectimax on what remains after the filter is applied. The safety filter predicts what the monsters are going to do in the very next step of the game, and

screens all moves to keep only the ones that survive after the predicted monster actions. After, the AI moves on to expectimax. The expectimax used weighs both the average outcome and the worst case outcome for any monster move, with 60% of the move's score being determined by the worst case outcome. This makes the AI's search more risk averse than an AI which picks moves by pure expected value, causing it to take longer to reach the goal but also have a lower chance of failing the stage and incurring a very bad score. Any tiebreakers in the search are determined by whichever move is going towards the exit, and then at random to avoid oscillating between two cells.

Our AI was tested on 5 different variant situations, with each variant being run 10 times. In the first variant, our AI is set in an empty world with no monsters. The world consists of empty, traversable squares, destroyable walls, and out of bounds areas. Additionally, there is an exit square that will count as the goal to reach. Once our AI paths to the exit, the game ends and our AI has won. This world is the base for the rest of the variants. In our testing, our AI was able to successfully path to the end and survive 100% of the time. Since there are no monsters, the AI is always in the SEEK_EXIT state, and just follows the A* path straight to the exit.

In the second variant, there is now a single "stupid" monster that will pick its next move at random every step. The start of the variant is the same as variant 1, with the AI following its A* path towards the exit. Once it nears the monster, though, it tries to maintain a safe distance while still trying to get closer to the exit. The monster's danger zone is wide enough that it isn't safe to pass it while it is in the middle of the open space, but as it randomly walks it eventually goes toward one of the edges of the world, and the AI has an opportunity to slip by it. After it passes the monster, it then switches back to A* to finish out the path to the exit. Testing this variant we also found our AI to have a 100% pass rate.

In the third variant, the monster changes from being “stupid” to being “self preserving”. Like the previous monster, it will go straight until it reaches an obstacle, then move randomly among the available moving cells. However, if the available moving cell would be an explosion, the monster would move away from it so as to not die. This keeps it so that the monster will keep itself alive, keeping the obstacle you have to pass from easily being removed. In our testing, we also found that our AI was able to survive 100% of the time.

The fourth variant makes another adjustment to the monster to make it “aggressive”, with it now able to sense and attack the AI from even further away, from 1 Chebyshev distance to 2. The start of the variant is the same yet again, and the AI maintains spacing from the monster just far enough away that the monster does not detect it. It does this until it finds an opportunity to slip past it to the exit like in previous iterations. This variant was the only one to test with a failure, having a 90% pass rate.

The fifth and final variant has a significant increase in difficulty, with it adding a “stupid” monster in addition to the “aggressive” one from variant 4. This makes it so that our AI will have to navigate two monsters to reach the exit, possibly having to deal with both at once. To pass, our AI had to find 2 opportunities to pass a monster to reach the exit, with the second of which being particularly dangerous due to it being stuck in between the monsters. If they move in just the wrong way, our AI could be stuck with no way to escape them. In our testing, though, it was able to find the right opportunities to sneak by each monster and escape 100% of the time. With this final variant tested, the results showed that our AI was very effective, and could survive all situations given to us over 50% of the time.

Testing showed that the AI was very consistent, reaching the goal almost every time across variants. There were a few main issues that could cause the AI to struggle, as it did in our

one failed run in variant 4. Starting with the variant 4 failure, the AI ended up dying to getting stuck between the monster and walls, limiting its movement options. This is a key theme with the AI, as many of the issues stem from the AI getting stuck between two things, with that being either a monster and a wall or between two separate monsters. In addition to these types of struggles, the AI also sometimes would have trouble trying to get past a monster when it lingered around the middle of the passageway, which would usually result in the AI trying to bomb the walls or the monster until it could find a way through. Often, this could happen near the exit, with the monster camping it and making it very difficult to escape. Despite these issues, overall our AI was very effective at playing these bomberman variants, with an overall pass rate of 98% across all variants.